

final关键字的理解

final关键字的理解

一、前言

1.1 前置知识点

- 1.变量初始化
- 2.常量
- 3.全局变量和局部变量的存储
- 4、编译期常量

二、final关键字详解

2.1 可修饰对象

1. 变量
 - (1) 基本数据类型
 - (2) 引用类型
 - (3) 扩展

3.修饰方法

4、修饰参数

5. 修饰类

2.2 final域重排序规则

2.3 扩展

1. final修饰的常量（编译期常量）在编译阶段会加入常量池（△）
2. 为什么局部/匿名内部类在使用外部局部变量时, 只能使用被final修饰的变量

结尾

1. 参考文章

一、前言

1.1 前置知识点

1.变量初始化

变量的初始化发生在以下几种情况：

- 直接初始化

在声明的同时初始化

```
final int x = 10;
```

- 构造函数中初始化

在类的构造函数中初始化的

```
public class Class {  
    final int x;  
  
    public MyClass(int value) {  
        x = value; // 初始化 x  
    }  
}
```

- 静态初始化

在静态块中进行初始化

```
class Class {
    static final int x;

    // 静态块
    static {
        x = 10; // 在静态初始化块中初始化
    }
}
```

- 代码块（实例初始化块）初始化

在实例初始化块中对 `final` 变量进行初始化

```
class Class {
    final int x;

    // 代码块
    {
        x = 10; // 实例初始化块
    }
}
```

2.常量

常量通常是指在程序运行期间其值不能被更改的变量，相关内容如下所示：

- 基本数据类型常量

在Java中，基本数据类型（如 `int`, `double`, `char`, `boolean` 等）可以被定义为常量，使用 `final` 关键字来声明一个常量。这意味着一旦赋值后，该变量的值就不能被修改。

```
final int MAX_SIZE = 100;
final double PI = 3.14159;
```

- 字符串常量

字符串是不可变的对象，因此可以将字符串字面量视为常量。字符串常量通常是指用 `final` 关键字声明的字符串变量，这种变量在初始化后不能再被改变。

```
final String GREETING = "Hello, world!";
```

注意：这里"Hello, World!"属于字符串字面量，其是固定的字符串值，属于常量，但是如果这里没有用 `final` 修饰，`GREETING` 就不是一个字符串常量，其是一个变量，只不过指向这个字符串字面量。

- 枚举常量

枚举类型是用来定义一组常量的类型。每个枚举值都是一个常量，可以用于表示一组固定的值

```
enum Color {
    RED, GREEN, BLUE
}
```

- 类常量

类常量是指用 `static` 和 `final` 修饰的变量，这使得它们属于类本身而不是类的实例。`static` 关键字表示该变量是属于类的，而不是某个特定实例；`final` 关键字则表示该变量的值在初始化后不能再被改变。

```
public class MathConstants {
    public static final double PI = 3.14159;
    public static final String GREETING = "Hello, World!";
}

public class Main {
    public static void main(String[] args) {
        // 通过类名直接访问类常量
        System.out.println("Pi: " + MathConstants.PI);
        System.out.println(MathConstants.GREETING);
    }
}
```

3.全局变量和局部变量的存储

关于类的结构信息、常量、静态变量和方法数据会存放在JVM的方法区中。

- 全局变量

全局变量在Java中通常是指类的静态变量或实例变量。

静态变量：存储在方法区（Method Area）。方法区是JVM的一部分，用于存放类的信息、常量、静态变量、即时编译器编译后的代码等。静态变量与类的生命周期相同，当类被加载时分配内存，程序结束时释放。

实例变量：存储在堆内存（Heap）。当一个对象被创建时，实例变量随对象一起存储在堆内存中。每个对象都有自己的实例变量，生命周期与对象相同。

```
public class A{
    private List<Integer> a = new ArrayList<>();
}
```

当你使用 `new A()` 创建一个 `A` 类的实例时，JVM首先会分配内存给这个对象。

然后，它会初始化实例变量。在你的例子中，`a` 会被初始化为 `new ArrayList<>()`。

最后，构造函数会被调用。

- 局部变量

当一个方法被调用时，JVM会为该方法在栈中分配一个新的栈帧（stack frame），局部变量（包括对象引用）会存储在这个栈帧中的局部变量表上。如果变量的类型是基本数据类型，则存在局部变量表上，如果是对象引用，其真正的对象数据存储在堆内存中（如果是基本数据类型则都存储在栈帧中），当局部变量是一个对象时，局部变量存储的是该对象的引用（reference），而不是对象本身的内容。引用是一个指向堆中对象内存地址的指针。

```
public void myMethod() {  
    MyObject obj = new MyObject(); // obj 是局部变量，存储在栈中  
}
```

`myMethod` 的栈帧创建，里面包含一个局部变量 `obj`，它存储的是指向堆中 `MyObject` 实例的引用，`new MyObject()` 在堆中分配内存，存储 `MyObject` 实例的属性和方法。

4、编译期常量

编译期常量（Compile-time constant）是在编译时就确定其值的常量。在Java中，编译期常量通常是使用 `final` 关键字声明的，并且必须是基本数据类型的常量，或者是字符串常量。只有在编译时能够确定值的常量才能被视为编译期常量。

参考文章：<https://juejin.cn/post/7016626301393960974>

二、final关键字详解

2.1 可修饰对象

`final`关键字可以修饰java中的三种对象（元素）：①变量 ②方法 ③类。

通俗理解：被`final`修饰的对象（元素）都不能够更改，但是不能更改对于不同的数据类型具有不同的含义。

注意：其实运行时常量，关于运行时常量和编译期常量可以参考前置知识点中的编译器常量。

1. 变量

（1）基本数据类型

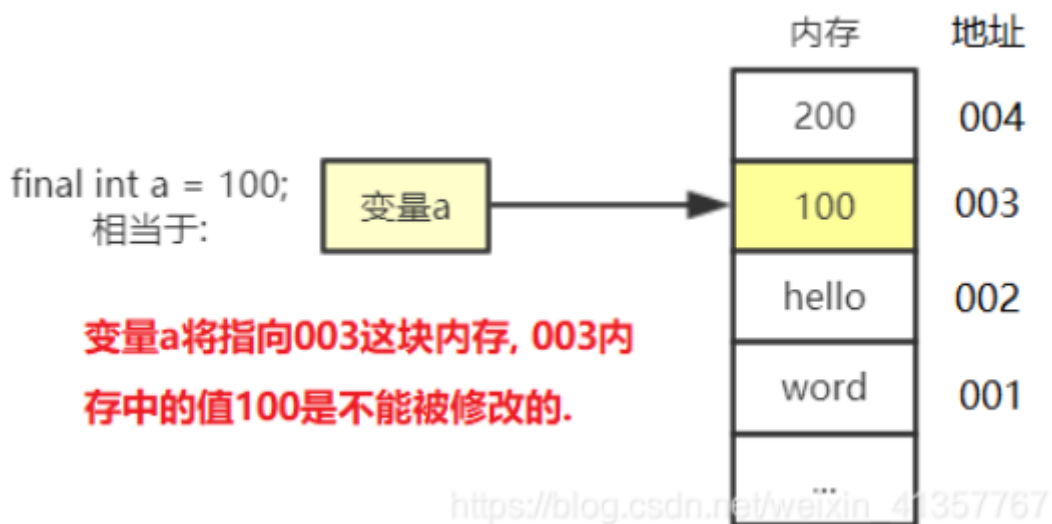
示例：

```
final int x = 10;
```

当`final`修饰的是一个基本数据类型数据时，这个数据的值在初始化（见1.1前置知识）后将不能被改变，具体示例如下所示

```
public static void main(String[] args) {  
    // 声明并初始化基本数据类型x的值  
    final int x = 1;  
    x = 2; // 编译报错（Cannot assign a value to final variable 'x' 不能给最终变量x赋值）  
}
```

关于`final`修饰基本数据类型时的内存示意图（来自参考文章）：



当变量a在初始化后, 将永远指向003这块内存, 而这块内存存在初始化之后就永远保存数值100不能够改变。

实际上, 变量a的存储位置根据它的作用域和声明位置而不同:

- 局部变量 (方法内定义的变量)

当 `x` 是方法中的局部变量时, 它会存储在**栈帧的局部变量表**中。每次调用方法时, JVM 会为该方法分配一个新的栈帧, 而局部变量 `x` 会在该栈帧中分配空间。方法执行完毕后, 栈帧被销毁, 局部变量也随之释放。

- 实例变量 (非静态全局变量)

如果 `x` 是类的实例变量 (非 `static` 修饰), 它会存储在**堆**中。每个类实例在堆上分配一个对象空间, 而 `x` 作为实例变量存储在该对象的内存空间中。不同实例的 `x` 变量相互独立, 各自持有自己的值。

- 静态变量 (静态全局变量)

如果 `x` 是类的静态变量 (即使用 `static` 修饰), 它会存储在**方法区 (Java 8 之后称为元空间)**中。静态变量是类级别的, 类加载时分配内存, 因此只在方法区中存储一个副本, 并且所有实例共享这个变量。

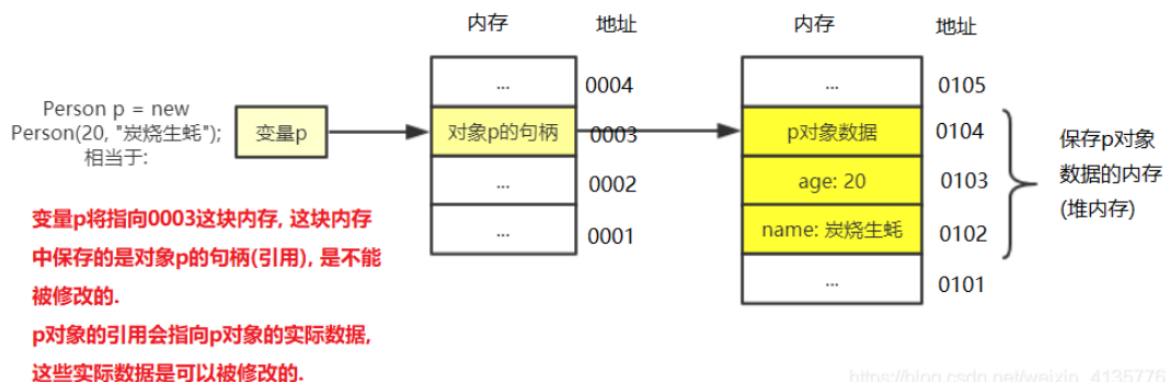
变量类型	存储位置	特点
局部变量	栈帧的局部变量表	方法调用时分配, 方法结束时销毁
实例变量	堆	每个对象实例独立持有自己的变量副本
静态变量	方法区/元空间 (Java 8+)	类级别变量, 所有对象共享一个变量副本

(2) 引用类型

示例:

```
final List<String> list = new ArrayList<>();
```

关于final修饰引用类型时的内存示意图 (来自参考文章):



变量p指向了0003这块内存, 0003内存中保存的是对象p的句柄(存放对象p数据的内存地址), 这个句柄值是不能被修改的, 也就是变量p永远指向p对象. 但是p对象的数据是可以修改的。

```
// 代码示例
public static void main(String[] args) {
    final Person p = new Person(20, "炭烧生蚝");
    p.setAge(18); //可以修改p对象的数据
    System.out.println(p.getAge()); //输出18

    Person pp = new Person(30, "蚝生烧炭");
    p = pp; //这行代码会报错, 不能通过编译, 因为p经final修饰永远指向上面定义的p对象, 不能指向pp对象.
}
```

final修饰变量的本质: final修饰的变量会指向一块固定的内存, 这块内存中的值不能改变。用类型变量所指向的对象之所以可以修改, 是因为引用变量不是直接指向对象的数据, 而是指向对象的引用的。所以被final修饰的引用类型变量将永远指向一个固定的对象, 不能被修改; 对象的数据值可以被修改。

(3) 扩展

- 所有的final修饰的字段都是编译期常量?

不是, 编译期常量是指: 在编译期就能确定的"常量", 编译期常量 = 运行期常量 (用final修饰的肯定是常量, 但这是针对运行期的, 准确的说是运行期常量) + 值是常量。

```
public class Test {
    //编译期常量
    final int i = 1;
    final static int j = 1;
    final int[] a = {1,2,3,4};
    //非编译期常量
    Random r = new Random();
    final int k = r.nextInt(); public final long c = System.currentTimeMillis();
    public static final long d = System.currentTimeMillis();
}
```

- static final 修饰必须在定义的时候进行赋值, 否则编译器将不予通过
- Java允许生成空白final, 也就是说被声明为final但又没有给出定值的字段, 但是必须在该字段被使用之前被赋值, 这给予我们两种选择
在定义处进行赋值(这不叫空白final)

在构造器中进行赋值，保证了该值在被使用前赋值。

```
public class Test {  
    final int i1 = 1;  
    final int i2; //空白final  
    public Test() {  
        i2 = 1;  
    }  
    public Test(int x) {  
        this.i2 = x;  
    }  
}
```

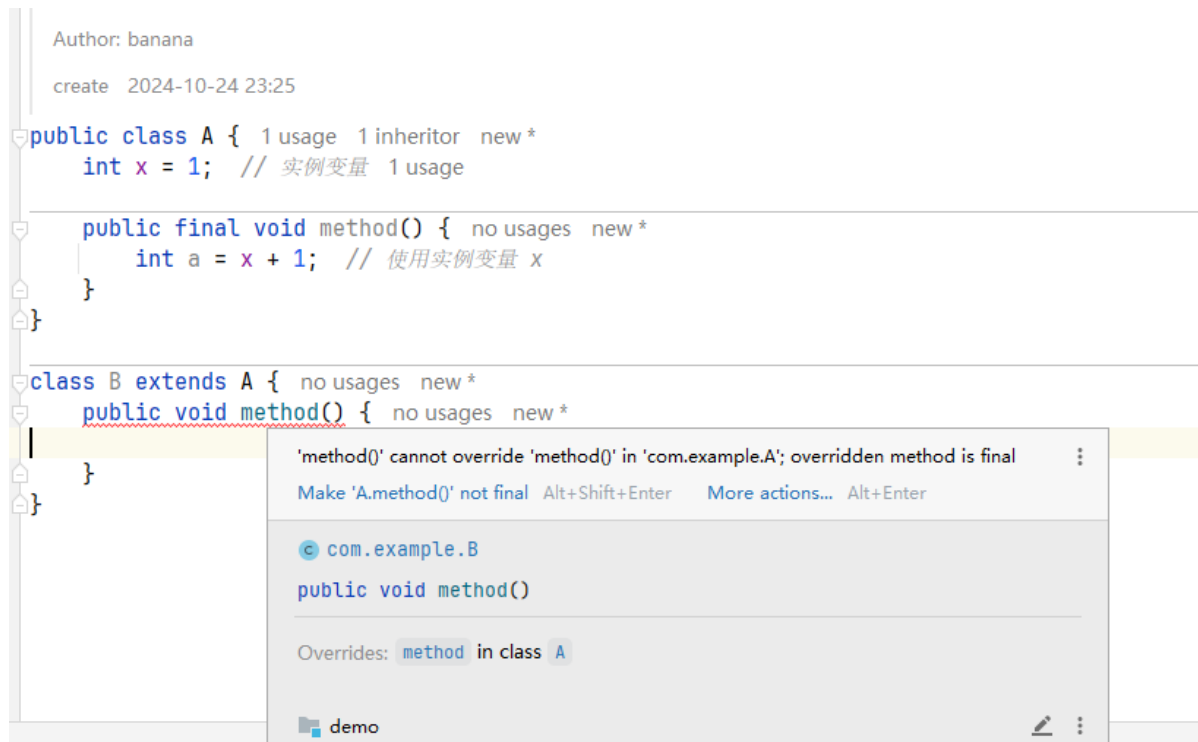
可以看到i2的赋值更为灵活。但是请注意，如果字段由static和final修饰，仅能在声明时赋值或声明后在静态代码块中赋值，因为该字段不属于对象，属于这个类。

3.修饰方法

示例：

```
class Parent {  
    public final void display() {  
        System.out.println("This is a final method.");  
    }  
}
```

- 被final修饰的方法不能够被子类重写



- final方法是可以被重载的

```

public class A {
    int x = 1; // 实例变量

    public final void method() {
        int a = x + 1; // 使用实例变量 x
    }

    public void method(String str) {
        System.out.println(str);
    }
}

```

- private方法是隐式的final

类中所有private方法都隐式地指定为final的，即private方法是隐式的 final，没有显式地使用 final 关键字修饰，private方法在编译时实际上被视作 final方法，private方法只在其声明的类中可见，子类无法访问，因此无法覆盖或重写 private方法。

重写private方法示例：

```

public class A {

    private void method() {
        System.out.println("A method");
    }

    public static void main(String[] args) {
        A a = new B();
        a.method(); // A method
    }

}

class B extends A {
    public void method() {
        System.out.println("B method");
    }
}

```

重写public方法示例：

```

public class A {

    public void method() {
        System.out.println("A method");
    }

    public static void main(String[] args) {
        A a = new B();
        a.method(); // B method
    }

}

```



```
class B extends A {  
    public void method() {  
        System.out.println("B method");  
    }  
}
```

4、修饰参数

Java允许在参数列表中以声明的方式将参数指明为final，这意味这你无法在方法中更改参数引用所指向的对象。

```
public class Example {  
  
    public void modifyValue(final String text) {  
        // text = "New value"; // 这会引发编译错误，因为 text 是 final 的  
        System.out.println("text: " + text);  
    }  
  
    public static void main(String[] args) {  
        Example example = new Example();  
        example.modifyValue("Original value");  
    }  
}
```

在上面的代码中，`text` 参数被声明为 `final`。这样就不允许在 `modifyValue` 方法内部重新赋值 `text`。如果你试图执行 `text = "New value";`，编译器会报错，因为 `final` 参数一旦传入方法就不能再指向其他对象。

5. 修饰类

被final修饰的类表示这个类是一个最终实现，不需要或不允许被进一步扩展。并且被final修饰的类所有成员方法都将被隐式修饰为final方法。

示例：

```
final class FinalClass {  
  
}
```

2.2 final域重排序规则

`final` 域的重排序规则主要是与Java内存模型（Java Memory Model, JMM）相关的，目的是确保在多线程环境中对 `final` 域的正确性和可见性。这些规则帮助解决了线程安全问题，尤其是在构造函数和对象共享时。

1. 重排序规则

在Java中，`final` 域具有以下重排序规则：

- **构造函数中的初始化**：当一个对象被创建并且构造函数执行时，任何 `final` 域的初始化都保证在该对象被引用之前完成。这意味着在构造函数中赋值给 `final` 域的操作是不可重排序的。

`final` 变量的赋值操作不能被重排序（即 `final` 变量必须在构造方法中完全初始化）。

构造函数执行完毕之前，其他线程无法看到这个对象的引用（不会看到未初始化的 `final` 变量）。

- **对象的可见性**：其他线程在看到对该对象的引用时，能够看到该对象的 `final` 域的正确初始化值。这意味着，即使对一个对象的引用在其构造完成之前被其他线程获取，`final` 域的值也会是可见的。
- 具体示例：

错误示例（未使用 `final`，可能导致对象未完全构造就被访问）

```
class UnsafeExample {
    private static UnsafeExample instance; // 没有 `final`
    private int value; // 可能被重排序

    public UnsafeExample() {
        value = 42; // 赋值可能被重排序
    }

    public static UnsafeExample getInstance() {
        if (instance == null) {
            instance = new UnsafeExample(); // 可能被其他线程提前看到
        }
        return instance;
    }

    public int getValue() {
        return value;
    }
}

public class UnsafeTest {
    public static void main(String[] args) {
        Runnable task = () -> {
            UnsafeExample instance = UnsafeExample.getInstance();
            System.out.println(Thread.currentThread().getName() + " 获取值："
+ instance.getValue());
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start();
        t2.start();
    }
}
```

可能的问题

由于指令重排序，`instance = new UnsafeExample()`；可能会被拆分成以下步骤：

分配内存

先让 `instance` 指向这块内存

执行构造函数 `UnsafeExample()`

赋值 `value = 42;`

如果线程 A 运行到第 2 步, `instance` 已经指向分配的内存, 但还没执行构造函数, 此时如果线程 B 调用 `getInstance()`, 就会获取到未完全构造的对象, `value` 可能仍是默认值 0 而不是 42, 导致错误行为。

正确示例 (使用 `final`, 保证构造完成后才发布)

```
class SafeExample {
    private static SafeExample instance;
    private final int value; // `final` 变量

    public SafeExample() {
        value = 42; // `final` 变量必须在构造方法内赋值
    }

    public static SafeExample getInstance() {
        if (instance == null) {
            instance = new SafeExample();
        }
        return instance;
    }

    public int getValue() {
        return value;
    }
}

public class SafeTest {
    public static void main(String[] args) {
        Runnable task = () -> {
            SafeExample instance = SafeExample.getInstance();
            System.out.println(Thread.currentThread().getName() + " 获取值: " + instance.getValue());
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start();
        t2.start();
    }
}
```

为什么 `final` 变量不会发生重排序?

JMM 规定 `final` 变量的特殊规则:

- 编译器禁止重排序 `final` 变量的初始化
 - `value = 42;` 必须在 `SafeExample()` 构造方法执行完之前完成, 不能被重排序到 `instance = new SafeExample();` 之后。
- 构造完成前, 对象的引用不会被发布
 - 只有当 `SafeExample` 完全构造完毕 (所有 `final` 变量初始化完成) 后, `instance` 才能被其他线程看到。

- 线程 B 不会获取到未初始化的 `final` 变量。

2. 解决的问题

这些重排序规则主要解决以下问题：

- **构造器安全性**：在多线程环境中，如果一个线程正在创建一个对象，另一个线程可能在对象完全构造完成之前就获取到了这个对象的引用，如下所示：

这种情况通常发生在 **重排序** 或 **发布未构造完成对象** 时，特别是在 **单例模式的双重检查锁** (Double-Checked Locking) 没有正确实现的情况下：

```
class Singleton {
    private static Singleton instance;

    private Singleton() {
        // 模拟构造过程
        try {
            Thread.sleep(1000); // 假设构造需要一些时间
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    public static Singleton getInstance() {
        if (instance == null) { // 第一次检查
            synchronized (Singleton.class) {
                if (instance == null) { // 第二次检查
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

当线程 A 正在构造 `Singleton` 对象时，线程 B 可能会在 `instance` 还未完全构造完成时就获取到了引用，从而导致使用了**未完全构造的对象**，可能会引发**空指针异常或其他不可预测的行为**。(因为指令重排的问题)

关于指令重排的详细说明：

`instance = new Singleton();` 这行代码并不是一个原子操作，它实际上分为三步：

- 分配内存
- 调用构造函数
- 将 `instance` 指向这块内存

由于**指令重排序**，可能导致执行顺序变成(示例)：

- 分配内存
- **先将 `instance` 指向这块内存（此时对象还未完全构造完成！）**
- 执行构造函数

下面的代码展示了多个线程竞争 `Singleton` 可能导致的问题：

```

public class SingletonTest {
    public static void main(String[] args) {
        Runnable task = () -> {
            Singleton instance = Singleton.getInstance();
            System.out.println(Thread.currentThread().getName() + " 获取
实例：" + instance);
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start();
        t2.start();
    }
}

```

如果此时t1方位的时候先只是分配了内存并先将 `instance` 指向这块内存，此时还是null值，当t2进行访问的时候，还会创建instance的实例对象。

这个问题在 **Java 1.5 之前** 的 **JMM** (Java 内存模型) 下尤为常见，因此在 **Java 1.5+** 版本中，**volatile** 关键字提供了更强的可见性和禁止重排序的能力。使用 **volatile** 关键字，禁止指令重排序：

```

class Singleton {
    private static volatile Singleton instance; // 添加 volatile

    private Singleton() {
        try {
            Thread.sleep(1000); // 模拟构造过程
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}

```

通过 **final** 域的重排序规则，确保了在对象构造完成之前，任何 **final** 域的初始化都是完整且可见的。

在Java 内存模型 (JMM) 中，**final 变量的初始化规则** 确保：

在构造函数中对 **final** 变量的赋值操作不会被重排序，即：

- **final 变量必须在构造函数执行期间完全初始化。**

一旦构造函数完成，其他线程看到该对象时，一定能看到 `final` 变量的正确值，即：

- `final` 变量的赋值在构造函数完成前不会被另一个线程看到。

如果 `Singleton` 的成员变量是 `final`，那么即使 `instance` 在构造过程中被另一个线程读取，该线程也至少能看到 `final` 变量被正确初始化。

```
class Singleton {
    private static Singleton instance;

    private final int value; // 使用 final 变量

    private Singleton() {
        this.value = 42; // `final` 变量必须在构造方法中赋值
        try {
            Thread.sleep(1000); // 模拟构造过程
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }

    public int getValue() {
        return value;
    }
}
```

为什么 `final` 可以防止未初始化对象被读取？

- **线程 A 先执行 `new Singleton()`**
 - `value` 作为 `final` 变量，必须在构造方法内初始化，并且不会被重排序。
 - **即使指令重排序** 使 `instance` 先指向分配的内存，其他线程读取时至少能保证 `value` 被正确初始化。
- **线程 B 获取 `instance`**，由于 `value` 是 `final`，JMM 保证了**线程 B 一定能看到 `value = 42`，不会看到未初始化的值。**

虽然 `final` 变量能保证它本身不会被重排序，但它 **不能** 解决整个对象发布过早的问题。例如，如果 `Singleton` 里还有**非 `final` 变量**，那么这些变量仍然可能被线程 B 看到 **未初始化** 的状态：

```
class Singleton {
    private static Singleton instance;
```

```

private final int value = 42; // `final` 变量保证初始化
private int otherValue; // 这个变量可能未被正确初始化

private Singleton() {
    otherValue = 100; // 这个赋值可能被重排序
}

public static Singleton getInstance() {
    if (instance == null) {
        synchronized (Singleton.class) {
            if (instance == null) {
                instance = new Singleton();
            }
        }
    }
    return instance;
}

public int getOtherValue() {
    return otherValue;
}
}

```

如果发生指令重排序：

1. `instance` 可能在 `otherValue = 100;` 之前被发布给另一个线程。
2. 线程 B 可能获取到 `instance`，但 `otherValue` 可能仍是默认值 `0`，而不是 `100`

解决方法：

1.使用 `volatile`

`volatile` 关键字不仅能防止指令重排序，还能确保可见性，保证所有变量都被正确初始化。

```
private static volatile Singleton instance;
```

2.使用静态内部类（推荐） 由于 JVM 规定 类的静态变量初始化是在类加载时完成的，它天然保证了线程安全

```

class Singleton {
    private Singleton() {}

    private static class Holder {
        private static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}

```

- **数据竞争：** `final` 域的使用可以减少数据竞争的风险，因为它们在初始化后不可更改，并且由于重排序规则，保证了在构造完成前不会被访问到不完全初始化的状态。

- **简化代码：**使用 `final` 关键字可以使代码在多线程环境中更简单，因为它减少了需要考虑的状态变化和同步机制。

示例

```
public class Example {
    private final int x;

    public Example(int value) {
        this.x = value; // 确保 x 的初始化完成
    }

    public int getX() {
        return x; // 其他线程在获取到对象引用后，可以安全地访问 x
    }
}
```

在这个示例中，`x` 是一个 `final` 域，任何对 `Example` 对象的引用都确保在构造完成后访问 `x`，从而避免了多线程中可能出现的未初始化状态。

总结：

通过这些重排序规则，Java 确保了 `final` 域在多线程环境中的安全性和可见性，有效避免了数据竞争和未初始化状态的问题。

方法	能否防止重排序？	能否保证线程安全？	适用场景
<code>final</code>	只防止 <code>final</code> 变量重排序	不能防止整个对象被过早发布	适用于单个 <code>final</code> 变量
<code>volatile</code>	完全防止重排序	能保证整个对象初始化后才被其他线程看到	适用于双重检查锁
静态内部类	JVM 类加载机制保证初始化顺序	线程安全	推荐

2.3 扩展

1. `final` 修饰的常量（编译期常量）在编译阶段会加入常量池（Δ）

`final` 是用于定义常量的，定义常量的好处是：不需要重复地创建相同的变量。而常量池是 Java 的一项重要技术，由 `final` 修饰的常量会在编译阶段放入到调用类的常量池中。


```

public static void main(String[] args) {
    int n1 = 2019;           //普通变量
    final int n2 = 2019;     //final修饰的变量

    String s = "20190522";
    String s1 = n1 + "0522"; //拼接字符串"20190512"
    String s2 = n2 + "0522";

    System.out.println(s == s1); //false
    System.out.println(s == s2); //true
}

```

整数-127-128是默认加载到常量池里的, 也就是说如果涉及到-127-128的整数操作, 默认在编译期就能确定整数的值. 所以这里我故意选用数字2019(大于128), 避免数字默认就存在常量池中.

- 首先根据final修饰的常量会在编译期放到常量池的原则, n2会在编译期间放到常量池中.
- 然后s变量所对应的"20190522"字符串会放入到字符串常量池中, 并对外提供一个引用返回给s变量.
- 这时候拼接字符串s1, 由于n1对应的数据没有放入常量池中, 所以s1暂时无法拼接, 需要等程序加载运行时才能确定s1对应的值.
- 但在拼接s2的时候, 由于n2已经存在于常量池, 所以可以直接与"0522"拼接, 拼接出的结果是"20190522". 这时系统会查看字符串常量池, 发现已经存在字符串20190522, 所以直接返回20190522的引用. 所以s2和s指向的是同一个引用, 这个引用指向的是字符串常量池中的20190522.
- 当程序执行时, n1变量才有具体的指向.
- 当拼接s1的时候, 会创建一个新的String类型对象, 也就是说字符串常量池中的20190522会对外提供一个新的引用.
- 所以当s1与s用"=="判断时, 由于对应的引用不同, 会返回false. 而s2和s指向同一个引用, 返回true.

总结: 这个例子想说明的是: 由于被final修饰的常量会在编译期进入常量池, 如果有涉及到该常量的操作, 很有可能在编译期就已经完成.

2. 为什么局部/匿名内部类在使用外部局部变量时, 只能使用被final修饰的变量

在JDK1.8以后, 通过内部类访问外部局部变量时, 无需显式把外部局部变量声明为final. 不是说不需要声明为final了, 而是这件事情在编译期间系统帮我们做了. 但是我们还是有必要了解为什么要用final修饰外部局部变量.

```

public class Outer {
    public static void main(String[] args) {
        final int a = 10;
        new Thread(){
            @Override
            public void run() {
                System.out.println(a);
            }
        }.start();
    }
}

```

在1.8以前, 上面这段代码, 如果没有给外部局部变量a加上final关键字, 是无法通过编译的. 可以试着想想: 当main方法已经执行完后, main方法的栈帧将会弹出, 如果此时Thread对象的生命周期还没有结束, 还没有执行打印语句的话, 将无法访问到外部的a变量.

通过javac编译得到.class文件(用IDE编译也可以), 然后在命令行输入 `javap -c .class文件的绝对路径`, 就能查看.class文件的反编译代码. 以上的Outer类经过编译产生两个.class文件, 分别是 `Outer.class` 和 `Outer$1.class`, 也就是说内部类会单独编译成一个.class文件. 下面给出 `Outer$1.class` 的反编译代码.

```
Compiled from "Outer.java"
final class forTest.Outer$1 extends java.lang.Thread {
    forTest.Outer$1();
    Code:
        0: aload_0
        1: invokespecial #1                  // Method java/lang/Thread."<init>":
()V
        4: return

    public void run();
    Code:
        0: getstatic     #2                  // Field
java/lang/System.out:Ljava/io/PrintStream;
        3: bipush        10
        5: invokevirtual #3                  // Method
java/io/PrintStream.println:(I)V
        8: return
}
```

定位到 `run()` 方法反编译代码中的第3行 (`3: bipush 10`) 我们看到a的值在内部类的 `run()` 方法执行过程中是以压栈的形式存储到本地变量表中的, 也就是说在内部类打印变量a的值时, 这个变量a不是外部的局部变量a, 因为如果是外部局部变量的话, 应该会使用 `load` 指令加载变量的值 (`load` 指令用于从局部变量表中加载变量的值. 如果变量是在外部类的方法中定义的, 内部类会使用 `load` 指令来访问它, 但由于局部变量的特殊性, 它访问的是局部变量的副本). 也就是说系统以拷贝的形式把外部局部变量a复制了一个副本到内部类中, 内部类有一个变量指向外部变量a所指向的值. 但研究到这里好像和final的关系还不是很大, 不加final似乎也可以拷贝一份变量副本, 只不过不能在编译期知道变量的值罢了. 这时该思考一个新问题了: 现在我们知道内部类的变量a和外部局部变量a是两个完全不同的变量, 那么如果在执行 `run()` 方法的过程中, 内部类中修改了a变量所指向的值, 就会产生数据不一致问题. 正因为我们的原意是内部类和外部类访问的是同一个a变量, 所以当在内部类中使用外部局部变量的时候应该用final修饰局部变量, 这样局部变量a的值就永远不会改变, 也避免了数据不一致问题的发生.

关于基本数据类型和引用数据类型的变量捕获:

- <https://blog.csdn.net/sjsjzhx/article/details/136976415>
- https://blog.csdn.net/weixin_71246590/article/details/139144066

在Java中, 内部类(包括匿名内部类)对局部变量的捕获处理有其特定的规则. 为了详细解释这一过程, 我们将根据 **基本数据类型** 和 **引用数据类型** 进行区分, 来说明Java内部类是如何捕获外部局部变量的

1. 内部类对局部变量的捕获

在Java中, 匿名内部类或任何形式的内部类要访问外部方法中的局部变量时, **局部变量必须是final** 或是 **有效的最终变量**. 这意味着在局部变量的生命周期内, 变量的值不能被修改.

为什么需要 `final`?

Java 需要确保内部类捕获的局部变量在内部类使用时是不可变的。如果局部变量值在外部方法中发生变化，内部类就会引用到一个可能已经改变了的变量，导致无法保证正确的行为。因此，局部变量被 **复制** 到匿名内部类中，并且其值在内部类的生命周期中是保持不变的。

对于基本数据类型和引用数据类型，Java 采取了不同的方式来捕获和存储这些局部变量。

2. 基本数据类型的捕获

当局部变量是 **基本数据类型** (如 `int`、`double`、`boolean` 等) 时，Java 编译器会将该变量的 **值** 复制到匿名内部类的实例字段中。

示例代码

```
public class Main {
    public static void main(String[] args) {
        int a = 10; // 基本数据类型
        Runnable r = new Runnable() {
            @Override
            public void run() {
                System.out.println(a); // 访问基本数据类型的局部变量
            }
        };
        r.run();
    }
}
```

编译后实现:

1. 编译器会生成一个匿名内部类，例如 `Main\1`，并为局部变量 `a` 创建一个 `final` 字段 (如 `a\0`)，该字段存储了 `a` 的值。
2. 在匿名内部类的 `run` 方法中，访问的是这个存储 `a` 值的字段，而不是原始局部变量 `a`。

```
final class Main\1 implements Runnable {
    private final int a\0; // 内部类字段，保存基本数据类型的副本

    Main\1(int a) {
        this.a\0 = a; // 将 a 的值传递给匿名类的构造函数
    }

    public void run() {
        System.out.println(this.a\0); // 访问匿名类中的字段
    }
}
```

结论:

- 对于基本数据类型，**值** 被捕获并存储在匿名内部类的一个成员字段中，这个字段是 `final` 的。
- 外部方法中局部变量的 **值** 被复制到匿名内部类的实例字段中，内部类通过该字段来访问该值。

3. 引用数据类型的捕获

当局部变量是 **引用数据类型**（如对象、数组等）时，Java 编译器会将该变量的 **引用** 复制到匿名内部类的实例字段中。

示例代码

```
public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Hello"); // 引用数据类型
        Runnable r = new Runnable() {
            @Override
            public void run() {
                System.out.println(sb); // 访问引用数据类型的局部变量
                sb.append(" world");
                System.out.println(sb); // 修改引用数据类型
            }
        };
        r.run();
    }
}
```

编译后实现：

1. 编译器会生成一个匿名内部类，并为局部变量 `sb` 创建一个 `final` 字段（如 `sb\0`），这个字段存储了对 `sb` 的引用。
2. 在匿名内部类中，访问的是该引用字段，因此可以对引用对象（如 `StringBuilder`）进行修改。

```
final class Main\1 implements Runnable {
    private final StringBuilder sb\0; // 内部类字段，保存引用数据类型的副本

    Main\1(StringBuilder sb) {
        this.sb\0 = sb; // 将 sb 的引用传递给匿名类的构造函数
    }

    public void run() {
        System.out.println(this.sb\0); // 访问匿名类中的字段
        sb\0.append(" world"); // 修改引用对象
        System.out.println(this.sb\0); // 打印修改后的对象
    }
}
```

结论：

- 对于引用数据类型，**引用** 被捕获并存储在匿名内部类的一个成员字段中，而不是对象本身。
- 内部类可以通过该引用字段修改对象的状态，因为引用本身指向的是堆上的同一个对象。

4. 总结

- **基本数据类型**：内部类捕获的是变量的 **值**，并将这个值复制到内部类的字段中。修改这个字段不会影响外部方法中的原始变量。
- **引用数据类型**：内部类捕获的是变量的 **引用**，并将这个引用复制到内部类的字段中。通过这个引用，内部类可以修改引用对象的状态（但不能改变引用本身）。

5. `final` 关键字的作用

- **对于基本类型：**`final` 确保局部变量的值不变，避免了在内部类中引用一个已经被修改过的局部变量的风险。
- **对于引用类型：**`final` 确保引用不能改变（即不能让引用指向另一个对象），但引用对象的内容可以被修改。

内部类的访问机制： 内部类可以访问外部类的成员，包括字段和方法，但对于局部变量的访问有一些特别的规则。Java 中的内部类不能直接访问外部类方法中的非 `final` 局部变量。在 Java 8 及以后的版本中，虽然局部变量可以是“有效的 final”但仍然需要遵循这个规则。

关于有效的final和声明的final说明

Java中，`final` 关键字有两种常见用法：一种是在声明时标记变量不可变，另一种则是在方法中表示变量不能被重新赋值。我们可以将它分为两个概念：**有效final** 和 **声明final**

- 声明final

当你声明一个变量时，如果用 `final` 关键字修饰它，这意味着变量的值一旦赋值后，就不能再改变了。对于基本数据类型，这意味着你不能修改变量的值；对于引用类型，这意味着你不能改变引用的对象，但可以改变对象的内容。

基础类型示例：

```
final int a = 10;
a = 20; // 编译错误，因为a是final变量，不能重新赋值
```

引用类型示例：

```
final List<String> list = new ArrayList<>();
list = new ArrayList<>(); // 编译错误，因为list是final引用，不能重新指向其他对象

list.add("Hello"); // 可以，因为是修改引用对象的内容
```

- 有效final

有效 `final` 是指，在代码中一个变量没有被显式声明为 `final`，但它的值在整个生命周期中也没有被修改过。在Java 8及之后的版本中，**有效 final** 变量可以用于局部内部类和Lambda表达式中。

换句话说，**有效 final** 变量的值在其生命周期内没有发生改变，尽管它没有加上 `final` 关键字。因此，JVM会将它视为一个 `final` 变量。

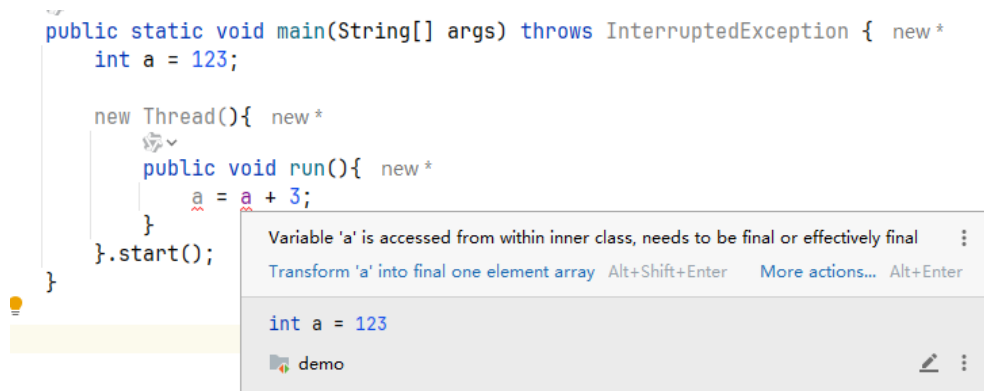
示例：

```
int a = 10;
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println(a); // 这里可以使用a，因为a在整个方法中没有被改变
    }
};
a = 20; // 如果a在这之后修改了，就不再是有效final了
```

如果你修改了 `a`，则它不再是有效 `final`，因为它的值发生了变化，导致你不能将它传递给内部类或者Lambda表达式

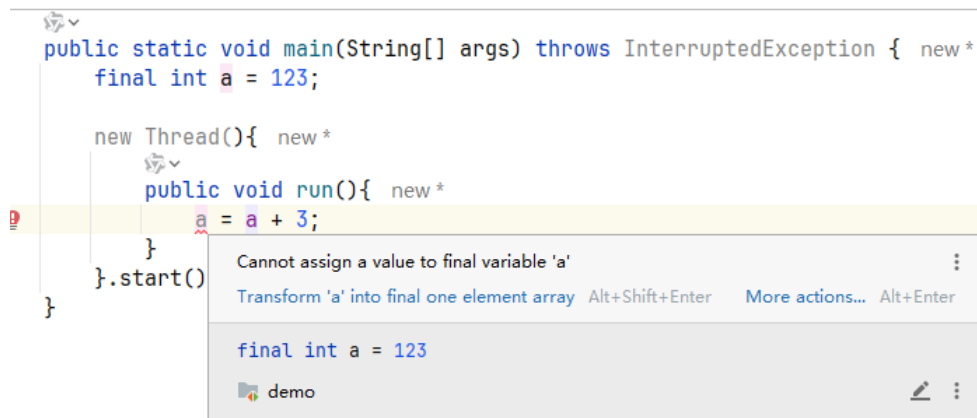
如果如下方式使用，首先会报需要将a声明为final，因为其值改变了，不在符合有效final

```
public static void main(String[] args) throws InterruptedException {  
    int a = 123;  
  
    new Thread(){  
        public void run(){  
            a = a + 3;  
        }  
    }.start();  
}
```



然后把final加上后，此时就会报基础类型在final修饰下，不能改变其值

```
public static void main(String[] args) throws InterruptedException {  
    final int a = 123;  
  
    new Thread(){  
        public void run(){  
            a = a + 3;  
        }  
    }.start();  
}
```



区别总结：

- **声明 final**：显式声明变量为 final，确保其值在初始化后不能再改变。

- **有效 final**：没有显式声明 `final`，但是变量在整个方法或代码块中值不被修改，JVM仍然把它当作 `final` 来处理，允许它在内部类、Lambda表达式等上下文中使用。

为什么要有有效final：

在Java 8及之后的版本中，`final` 和**有效 final** 变量都可以被传递到Lambda表达式中或被内部类捕获。这是为了确保变量的值在这些上下文中保持一致，避免可能的线程安全问题

具体几种示例：

```
public class A {

    public static void main(String[] args) throws InterruptedException {
        final B b = new B();
        b.setName("123");

        new Thread(){
            public void run(){
                System.out.println("Thread value:" + b);
                b.setName("abc");
                System.out.println("Thread value:" + b);
            }
        }.start();

        Thread.sleep(10);
        System.out.println("main Thread value:" + b);

    }

}

@Data
class B {

    private String name;

}
```

结果如下，因为其是引用类型，虽然加了final，但是只要不改变其引用地址即可，可以改变引用对象中的内容

```
Thread value:B(name=123)
Thread value:B(name=abc)
main Thread value:B(name=abc)
```

如果是基本数据类型，那么由于其被final修饰，是不允许改变的

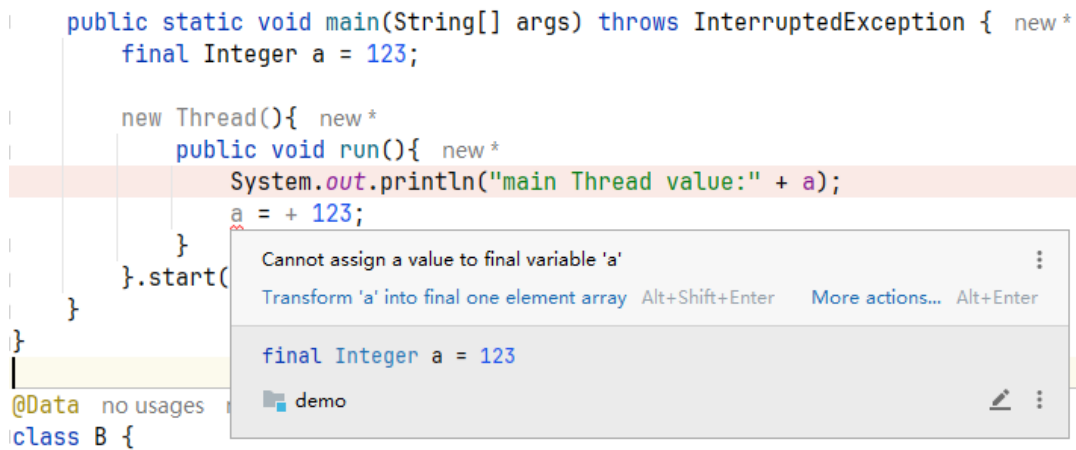
```

public class A {

    public static void main(String[] args) throws InterruptedException {
        final Integer a = 123;

        new Thread(){
            public void run(){
                System.out.println("main Thread value:" + a);
                a = + 123;
            }
        }.start();
    }
}

```



在 `a + 123` 时，首先会把 `a` 的值解包成一个基本类型 `int`，然后进行加法运算，接着会创建一个新的 `Integer` 对象并将新的结果值其赋值给 `a`，此时引用地址改变了，不符合 `final` 不改变的要求。

结尾

1. 参考文章

final 相关

- <https://www.cnblogs.com/tanshaoshenghao/p/10908771.html>
- https://blog.csdn.net/qq_24309787/article/details/100942044
- <https://pdai.tech/md/java/thread/java-thread-x-key-final.html>

编译器常量

- <https://juejin.cn/post/7016626301393960974>